
Don’t Overthink it. Preferring Shorter Thinking Chains for Improved LLM Reasoning

Michael Hassid^{1,2}, Gabriel Synnaeve¹, Yossi Adi^{1,2}, Roy Schwartz²

¹FAIR Team, Meta ²The Hebrew University of Jerusalem

Abstract

Reasoning large language models (LLMs) heavily rely on scaling test-time compute to perform complex reasoning tasks by generating extensive “thinking” chains. While demonstrating impressive results, this approach incurs significant computational costs and inference time. In this work, we challenge the assumption that long thinking chains results in better reasoning capabilities. We first demonstrate that shorter reasoning chains within individual questions are significantly more likely to yield correct answers—up to 34.5% more accurate than the longest chain sampled for the same question. Based on these results, we suggest *short- $m@k$* , a novel reasoning LLM inference method. Our method executes k independent generations in parallel and halts computation once the first m thinking processes are done. The final answer is chosen using majority voting among these m chains. Basic *short-1@k* demonstrates similar or even superior performance over standard majority voting in low-compute settings—using up to 40% fewer thinking tokens. *short-3@k*, while slightly less efficient than *short-1@k*, consistently surpasses majority voting across all compute budgets, while still being substantially faster (up to 33% wall time reduction). Inspired by our results, we finetune an LLM using short, long, and randomly selected reasoning chains. We then observe that training on the shorter ones leads to better performance. Our findings suggest rethinking current methods of test-time compute in reasoning LLMs, emphasizing that longer “thinking” does not necessarily translate to improved performance and can, counter-intuitively, lead to degraded results.

1 Introduction

Scaling test-time compute has been shown to be an effective strategy for improving the performance of reasoning LLMs on complex reasoning tasks [OpenAI, 2024, 2025, Team, 2025a]. This method involves generating extensive *thinking*—very long sequences of tokens that contain enhanced reasoning trajectories, ultimately yielding more accurate solutions. Prior work has argued that longer model responses result in enhanced reasoning capabilities [Guo et al., 2025, Muennighoff et al., 2025, Anthropic, 2025]. However, generating such long-sequences also leads to high computational cost and slow decoding time due to the autoregressive nature of LLMs.

In this work, we demonstrate that scaling test-time compute does not necessarily improve model performance in the way previously thought. We start with a somewhat surprising observation. We take three leading reasoning LLMs, and for each generate multiple answers to each question in three complex math benchmarks. We then observe that taking the shortest answer for each question strongly and consistently outperforms both selecting a random answer (up to 18.8% gap) and taking the longest one (up to 34.5% gap). These performance gaps are on top of the natural reduction in sequence length—the shortest chains are 50% and 67% shorter than the random and longest chains respectively.

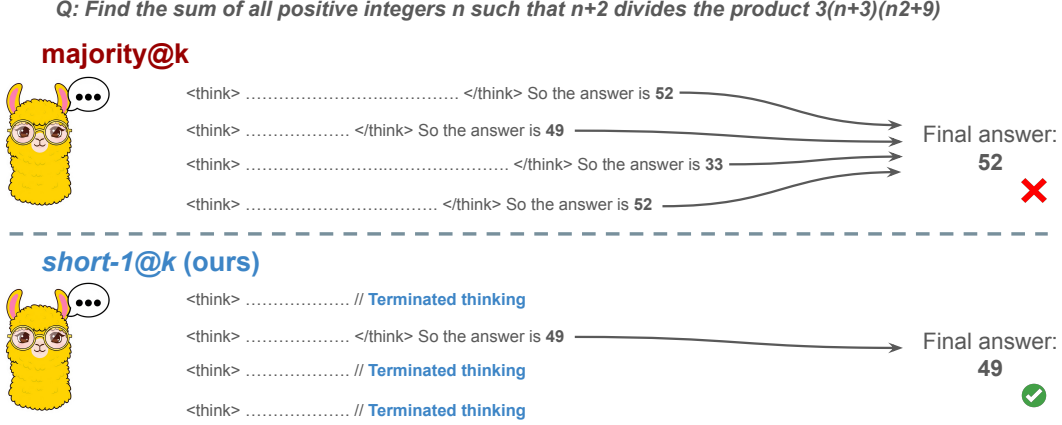


Figure 1: Visual comparison between majority voting and our proposed method *short-m@k* with $m = 1$ (“...” represent thinking time). Given k parallel attempts for the same question, *majority@k* waits until all attempts are done, and perform majority voting among them. On the other hand, our *short-m@k* method halts computation for all attempts as soon as the first m attempts finish “thinking”, which saves much compute and time, and surprisingly also boost performance in most cases.

Building on these findings, we propose *short-m@k*—a novel inference method for reasoning LLMs. *short-m@k* executes k generations in parallel and terminates computation for all generations as soon as the first m thinking processes are completed. The final answer is then selected via majority voting among those shortest chains, where ties are broken by taking the shortest answer among the tied candidates. A high-level, visual description of the proposed method, can be seen on Figure 1.

We evaluate *short-m@k* using three leading reasoning LLMs, and compare it to majority voting [Wang et al., 2022]—the most common aggregation method for evaluating reasoning LLMs on complex benchmarks [Abdin et al., 2025]. We show that in low-compute regimes, *short-1@k*, i.e., taking the single shortest chain, outperforms majority voting, while significantly reducing the time and compute needed to generate the final answer. For example, using LN-Super-49B [Bercovich et al., 2025], *short-1@k* can reduce up to 40% of the compute while giving the same performance as majority voting. Moreover, for high-compute regimes, *short-3@k*, which halts generation after three thinking chains are completed, consistently outperforms majority voting across all compute budgets, while running up to 33% faster.

Based on our findings, we study whether training on short reasoning chains can lead to more accurate models. To do so, we finetune the Qwen-2.5-32B model [Yang et al., 2024] on three variants of the S1 dataset [Muennighoff et al., 2025]: S1-short, S1-long, and S1-random, consisting of examples with the shortest, longest, and randomly sampled reasoning trajectories among several generations, respectively. Our experiments demonstrate that finetuning Qwen-2.5-32B on S1-short not only yields shorter thinking lengths, but also improves model performance. Conversely, finetuning on S1-long increases reasoning time with no significant performance gains.

This work rethinks the test-time compute paradigm for reasoning LLMs, showing that longer thinking not only does not ensure better reasoning, but also leads to worse reasoning in most cases. Our *short-m@k* methods prioritize shorter reasoning, yielding improved performance and reduced computational costs for current reasoning LLMs. We also show that training reasoning LLMs with shorter reasoning trajectories can enhance performance and reduce costs. Our results pave the way towards a new era of efficient and high-performing reasoning LLMs.

2 Related work

Reasoning LLMs and test-time scaling. Reasoning LLMs tackle complex tasks by employing extensive reasoning processes, often involving detailed, step-by-step trajectories [OpenAI, 2024, 2025, Team, 2025a, Abdin et al., 2025, Anthropic, 2025, Bercovich et al., 2025, Guo et al., 2025, sky, 2025, DeepMind, 2025, Team, 2025b]. This capability is fundamentally based on techniques like

chain-of-thought [CoT; Wei et al., 2022], which encourages models to generate intermediate reasoning steps before arriving at a final answer. By using a greater number of tokens to explore multiple problem-solving approaches, employ self-reflection, and perform verification, reasoning LLMs have demonstrated superior performance on challenging tasks such as mathematical problem-solving and code generation [Ke et al., 2025].

The ability of extensive chain-of-thought capabilities in LLMs is typically achieved through post-training methods applied to a strong base model. The two primary approaches for instilling or improving this reasoning ability are the application of reinforcement learning (RL) [Guo et al., 2025, Team, 2025a] and supervised fine-tuning [Muennighoff et al., 2025, Ye et al., 2025].

Guo et al. [2025] have demonstrated that as training progresses the model tends to generate longer thinking trajectories, which result in improved performance on complex reasoning tasks. Similarly, Anthropic [2025] and Muennighoff et al. [2025] have shown a correlation between increased average thinking length during inference and improved model performance. In this study, we challenge this assumption, demonstrating that shorter sequences are more likely to provide the correct answer.

Efficiency in reasoning LLMs. While shortening the length of CoT is beneficial for non-reasoning models [Nayab et al., 2024, Kang et al., 2025], it is highly important for reasoning LLMs as they require a very large amount of tokens to perform the thinking process. As a result, very recent studies tried to make the process more efficient, e.g., by using early exit techniques for reasoning trajectories [Pu et al., 2025, Yang et al., 2025], or by training reasoning models which enable control over the thinking length [Yu et al., 2025].

Several concurrent works studied the relationship between reasoning trajectory length and correctness. Lu et al. [2025] proposed a method for reducing the length of thinking trajectories in reasoning training datasets. Their method employs a reasoning LLM several times over an existing trajectory in order to make it shorter. As this approach eventually trains a model over shorter trajectories it is similar to the method we employ in Section 5. However, our method is simpler as it does not require an LLM to explicitly shorten the sequence. Fatemi et al. [2025] and Arora and Zanette [2025] proposed an RL methods to shorten reasoning in language models. Fatemi et al. [2025] also observed that correct answers typically require shorter thinking trajectories by averaging lengths across examples, suggesting that lengthy responses might inherently stem from RL-based optimization during training. In Section 3 we show that indeed correct answers usually use longer thinking trajectories, but also highlight that averaging across all examples might hinder this effect as easier questions require sustainably lower amount of reasoning tokens compared to harder ones. Wang et al. [2025] conducted a comprehensive study on reasoning trajectories. They found that for a specific question, correct responses from reasoning models are usually shorter than incorrect ones. We provide further analysis supporting this observation in Section 3. Moreover, our proposed inference method *short-m@k* is designed to enhance the efficiency of reasoning LLMs by leveraging this property.

3 Shorter thinking is preferable

As mentioned above, the common wisdom in reasoning LLMs suggests that increased test-time computation enhances model performance. Specifically, it is widely assumed that longer reasoning process, which entails extensive reasoning thinking chains, correlates with improved task performance [OpenAI, 2024, Anthropic, 2025, Muennighoff et al., 2025]. We challenge this assumption and ask whether generating more tokens actually leads to better performance. To that end, we generate multiple answers per question and compare performance based solely on the shortest, longest and randomly sampled thinking chains among the generated samples.

3.1 Experimental details

We consider three leading, high-performing, open, reasoning LLMs. Llama-3.3-Nemotron-Super-49B-v1 [LN-Super-49B; Bercovich et al., 2025]: a reasoning RL-enhanced version of Llama-3.3-70B [Grattafiori et al., 2024]; R1-Distill-Qwen-32B [R1-32B; Guo et al., 2025]: an SFT finetuned version of Qwen-2.5-32B-Instruct [Yang et al., 2024] derived from R1 trajectories; and QwQ-32B a reasoning RL-enhanced version Qwen-2.5-32B-Instruct [Team, 2025a].

We evaluate all models using three competitive reasoning benchmarks: AIME 2024 [of America, 2024], AIME 2025 [of America, 2025] and HMMT February 2025, from the Math Arena benchmark [Balunović et al., 2025]. The three benchmarks are derived from math competitions, and involve solving problems that cover a broad range of mathematics topics. Each dataset consists of 30 examples with varied difficulty.

For each question, we generate 20 responses per model, yielding a total of 5,400 generations. For all models we use temperature of 0.7, top-p=0.95 and a maximum number of generated tokens of 32,768. When measuring the thinking chain length, we measure the token count between the <think> and </think> tokens. We run inference for all models using paged attention via the vLLM framework [Kwon et al., 2023].¹

3.2 The shorter the better

We first note that as observed in recent studies [Anthropic, 2025, OpenAI, 2024, Muennighoff et al., 2025], thinking chains tend to be longer for harder questions. To quantify this phenomenon with our generated samples, for each model we split the questions into three equal size groups according to model’s success rate. Then, we calculate the average thinking length per easier and harder questions.² We also provide the average lengths for the correct and incorrect attempts per split.

Table 1 shows that indeed models use more tokens for more challenging questions, up to a factor of 2.9. This may lead to the assumption that longer thinking chains leads to more complex reasoning, and therefore, better performance. Nevertheless, surprisingly, we also observe that, within each question subset, correct answers are typically *shorter* than incorrect ones. To study the connection between performance and thinking length in a controlled manner, we turn to compare different answers *to the same question*.

Table 1: Average thinking tokens for correct (C), incorrect (IC) and all (A) answers, per easier and harder questions. The numbers are in thousands of tokens.

Model	Easy C/IC/A	Hard C/IC/A
LN-Super-49B	5.3/11.1/5.7	12.4/16.8/16.6
R1-32B	4.9/13.7/5.3	14.4/15.8/15.7
QwQ-32B ³	8.4/ – /8.4	19.1/22.8/22.3

We compare *short* vs. *long* thinking chains for the same question, along with a random chain. Results are presented in Table 2. First, as expected, the shortest answers are 25%–50% shorter compared to randomly sampled responses. However, we also note that across almost all models and benchmarks, considering the answer with the shortest thinking chain actually boosts performance, yielding an average absolute improvement of 2.2%–15.7% across benchmarks compared to randomly selected generations. When considering the longest thinking answers among the generations, we further observe an increase in thinking chain length, with up to 75% more tokens per chain. These extended reasoning trajectories substantially degrade performance, resulting in average absolute reductions ranging between 12%–18.8% compared to random generations. These trends are most noticeable when comparing the shortest generation with the longest ones, with an absolute performance gain of up to 34.5% in average accuracy and a substantial drop in the number of thinking tokens.

The above results suggest that long generations might come with a significant price-tag, not only in running time, but also in performance. While more complex questions generally require a greater number of thinking tokens, within an individual example, shorter thinking trajectories are much more likely to be correct. This observation challenges the assumption for reasoning LLMs that longer reasoning processes typically improve performance. Next, we propose strategies to leverage these findings to improve the efficiency and effectiveness of reasoning LLMs.

4 *short-m@k*: faster and better inference of reasoning LLMs

Based on the results presented in Section 3, we suggest a novel inference method for reasoning LLMs. Our method—*short-m@k*—leverages batch inference of LLMs per question, using multiple

¹We used 8 H100 GPUs per model. The total decoding time for all models sums up to about 110 hours.

²In this section we exclude generations where thinking is not completed within the maximum generation length, as these often result in an infinite thinking loop.

³The QwQ-32B model correctly answered all of its easier questions in all attempts.

Table 2: Shorter thinking performs better. Comparison between taking the shortest/longest/random generation per example.

	AIME 2024		AIME 2025		HMMT		Average	
	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑
LN-Super-49B								
random	11258	58.8	12105	51.3	13445	33.0	12270	47.7
longest	18566	33.3	18937	30.0	19790	23.3	19098 (+56%)	28.9
shortest	6276	76.7	7036	66.7	7938	46.7	7083 (−42%)	63.4
R1-32B								
random	9614	71.8	11558	56.4	12482	38.3	11218	55.5
longest	17689	53.3	19883	36.7	20126	23.3	19233 (+71%)	37.8
shortest	4562	80.0	6253	63.3	6557	36.7	5791 (−48%)	60.0
QwQ-32B								
random	13093	82.0	14495	72.3	16466	52.5	14685	68.9
longest	20059	70.0	21278	63.3	24265	36.7	21867 (+49%)	56.7
shortest	8655	86.7	10303	66.7	11370	60.0	10109 (−31%)	71.1

parallel decoding runs for the same query. We begin by introducing our method in Section 4.1. We then describe our evaluation methodology, which takes into account inference compute and running time (Section 4.2). Finally, we present our results (Section 4.3) and ablation studies (Section 4.4).

4.1 The *short-m@k* method

The *short-m@k* method, visualized in Figure 1, performs parallel decoding of k generations for a given question, halting computation across all generations as soon as the $m \leq k$ shortest thinking trajectories are completed. It then conducts majority voting among those shortest answers, resolving ties by selecting the answer with the shortest thinking chain. Given that thinking trajectories can be computationally intensive, terminating all generations once the m shortest trajectories are completed not only saves computational resources but also significantly reduces wall time due to the parallel decoding approach, as shown in Section 4.3.

Below we focus on *short-1@k* and *short-3@k*, with *short-1@k* being the most efficient variant of *short-m@k* and *short-3@k* providing the best balance of performance and efficiency (see Section 4.3). Ablation studies on m and other design choices are presented in Section 4.4.

4.2 Evaluation setup

We evaluate all methods under the same setup as described in Section 3.1. We report results using our method (*short-m@k*) with $m \in \{1, 3\}$. We compare the proposed method to the standard majority voting (*majority@k*), arguably the most common method for aggregating multiple outputs [Wang et al., 2022], which was recently adapted for reasoning LLMs [Guo et al., 2025, Abdin et al., 2025, Wang et al., 2025]. As an oracle, we consider *pass@k* [Kulal et al., 2019, Chen et al., 2021], which measures the probability of including the correct solution within k generated responses.

We benchmark the different methods with sample sizes of $k \in \{1, 2, \dots, 10\}$, assuming standard parallel decoding setup, i.e., all samples are generated in parallel. For the oracle (*pass@k*) approach, we use the unbiased estimator presented in Chen et al. [2021], with our 20 generations per question ($n=20$). For the *short-1@k* method, we use the *rank-score@k* metric [Hassid et al., 2024], where we sort the different generations according to thinking length. For *majority@k* and *short-m@k* where $m > 1$, we run over all k -sized subsets out of the 20 generations per example.

We evaluate the different methods considering three main criteria: (a) *Sample-size* (i.e., k), where we compare methods while controlling for the number of generated samples; (b) *Thinking-compute*, where we measure the total number of thinking tokens used across all generations in the batch; and (c) *Time-to-answer*, which measures the wall time of running inference using each method. We note that we assume that currently served LLMs operate with a fixed processing speed in a fixed batch

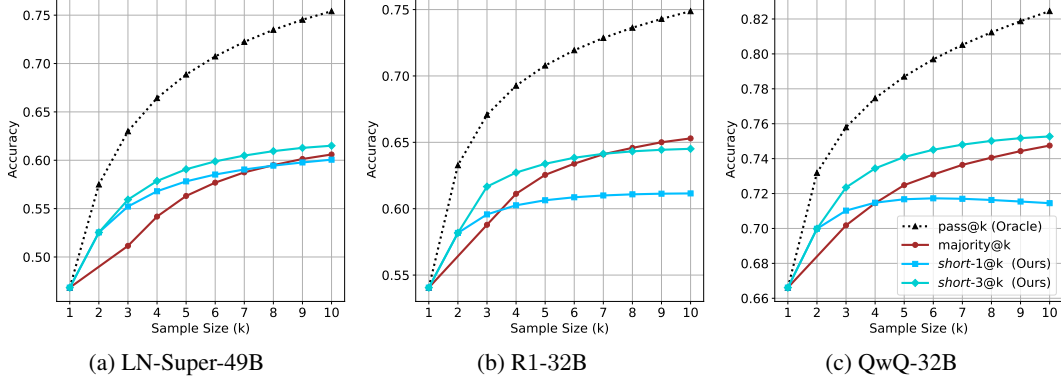


Figure 2: Comparing different inference methods under controlled sample size (k). All methods improve with larger sample sizes. Interestingly, this trend also holds for the *short- m @ k* methods.

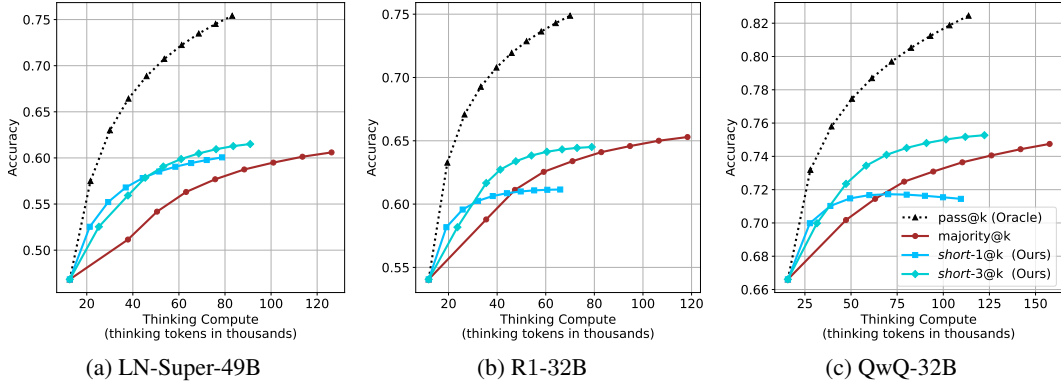


Figure 3: Comparing different inference methods under controlled thinking compute. *short-1@k* is highly performant in low compute regimes. *short-3@k* dominates the curve compared to *majority@k*.

setting while using parallel decoding. As a result, using our method (*short- m @ k*), we terminate all other generations after the first m decoding thinking processes terminate. Thus, the overall thinking compute is the total number of thinking tokens for each of the k generations at that point. Similarly, the overall time is that of the m 'th shortest generation process. Conversely, for *majority@k*, the method's design necessitates waiting for all generations to complete before proceeding. Hence, we consider the compute as the total amount of thinking tokens in all generations and run time according to the longest thinking chain. As for the oracle approach, we terminate all thinking trajectories once the shortest correct one is finished, and consider the compute and time accordingly.

4.3 Results

Sample-size (k). We start by examining different methods across benchmarks for a fixed sample size k . Results aggregated across benchmarks are presented in Figure 2, and detailed results per benchmark can be seen at Appendix A. We observe that, generally, all methods improve with larger sample sizes, indicating that increased generations per question enhance performance. This trend is somewhat expected for the oracle (*pass@k*) and *majority@k* methods but surprising for our method, as it means that even when a large amount of generations is used, the shorter thinking ones are more likely to be correct. The only exception is QwQ-32B (Figure 2c), which shows a small decline when considering larger sample sizes with the *short-1@k* method.

When comparing *short-1@k* to *majority@k*, the former outperforms at smaller sample sizes, but is outperformed by the latter in two out of three models when the sample size increases. Meanwhile, the *short-3@k* method demonstrates superior performance, dominating across nearly all models and sample sizes. We next analyze how this performance advantage translates into efficiency benefits.

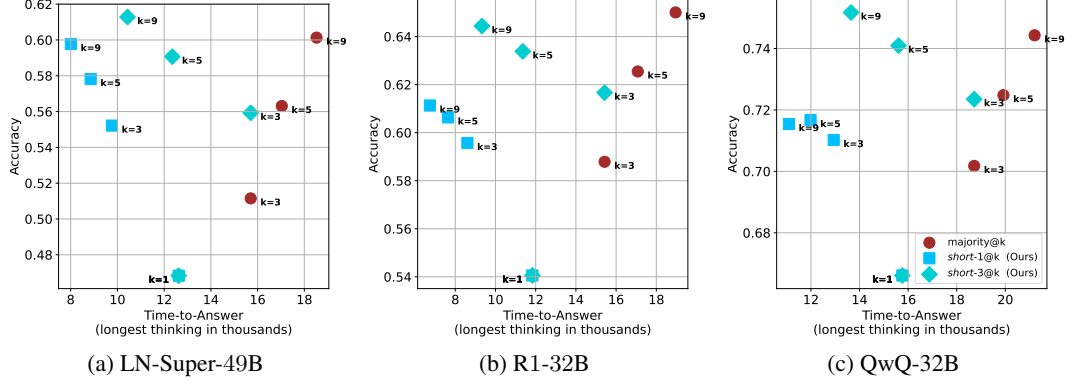


Figure 4: Comparing time-to-answer for different inference methods. Our methods substantially reduce time cost with no major loss in performance. $short-3@k$ improves performance across all models and sample sizes. We note that unlike $majority@k$, which becomes slower as k grows, our methods run **faster** with k , as the probability of finding a short chain increases with k .

Thinking-compute. The aggregated performance results, evaluated with respect to thinking compute, are presented in Figure 3 (per-benchmark results provided in Appendix A). We again observe that the $short-1@k$ method outperforms $majority@k$ at lower compute budgets. Notably, for LN-Super-49B (Figure 3a), the $short-1@k$ method surpasses $majority@k$ across all compute budgets. For instance, $short-1@k$ achieves 57% accuracy with approximately 60% of the compute budget used by $majority@k$ to achieve the same accuracy. For R1-32B and QwQ-32B models, the $short-1@k$ method exceeds $majority@k$ up to compute budgets of 45k and 60k total thinking tokens, respectively, but is underperformed by it on larger compute budgets.

The $short-3@k$ method yields even greater performance improvements, incurring only a modest increase in thinking compute compared to $short-1@k$. When compared to $majority@k$, $short-3@k$ consistently achieves higher performance with lower thinking compute across all models and compute budgets. For example, with the QwQ-32B model (Figure 3c), and an average compute budget of 80k thinking tokens per example, $short-3@k$ improves accuracy by 2% over $majority@k$.

Time-to-answer. Finally, the aggregated time-to-answer results are shown in Figure 4, with per-benchmark details in Appendix A.⁴ As sample size increases, $majority@k$ exhibits longer time-to-answer, driven by a higher probability of sampling generations with extended thinking chains, requiring all trajectories to complete. Conversely, the $short-1@k$ method shows *reduced* time-to-answer with larger sample sizes, as the probability of encountering a short answer increases. This trend also holds for the $short-3@k$ method after three reasoning processes complete.

This phenomenon makes the $short-1@k$ and $short-3@k$ methods substantially more usable compared to basic $majority@k$. For example, when using the LN-Super-49B model (Figure 4a), with a sample size of 5, the $short-1@k$ method reduces time consumption by almost 50% while also increasing performance by about 1.5% compared to $majority@k$. When considering a larger sample size of 9, the performance values are almost the same but $short-1@k$ is more than 55% faster.

Finally, we observe that for most models and sample sizes, $short-3@k$ boosts performance, while for larger ones it also reduces time-to-answer significantly. For example, on R1-32B (Figure 4b), with $k = 5$, $short-3@k$ is 33% faster than $majority@k$, while reaching superior performance. A similar boost in time-to-answer and performance is observed with QwQ-32B and sample size 9 (Figure 4c).

4.4 Ablation studies

We investigate two axis of $short-m@k$: the value of m and the tie breaking method. For all experiments we use LN-Super-49B, reporting results over the three benchmarks described in Section 3.1. For the ablation studies we focus on controlling thinking compute.

⁴For readability, the oracle is excluded from Figure 4, and methods are compared across a subset of sample sizes.

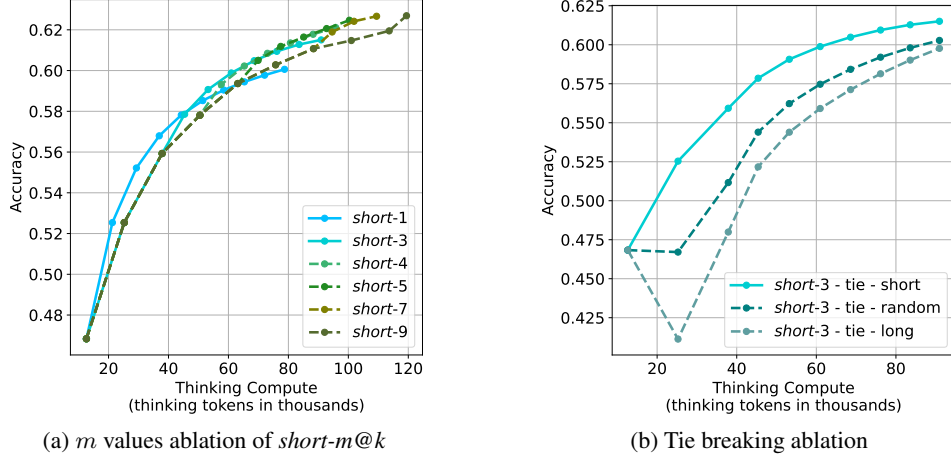


Figure 5: Ablation studies over different m values for $short-m@k$, and different tie breaking methods. Both figures show the model’s average accuracy across benchmarks as a function of the length of its thinking trajectories (measured in thousands).

We start by ablating different $m \in \{1, 3, 4, 5, 7, 9\}$ for $short-m@k$. Results are shown in Figure 5a. As observed in our main results, $short-1@k$ outperforms others in low-compute regimes, while being less effective for larger compute budgets. Larger m values seem to perform similarly, with higher m values yielding slightly better results in high-compute scenarios.

Next, we analyze the tie-breaking rule of $short-m@k$. We suggest the selection of the shortest reasoning chain among the vote-leading options. We compare this strategy to random tie-breaking, and to tie breaking according to the longest reasoning chain among the options. As shown in Figure 5b, the $short-m@k$ strategy outperforms random tie-breaking. In contrast, choosing the option with the longest reasoning chain yields inferior results.

5 Finetuning using shorter trajectories

Drawing from our findings, we investigate whether fine-tuning on shorter reasoning chains enhances the accuracy of reasoning in LLMs. To do so, we follow the S1 paradigm, which fine-tunes an LLM to perform reasoning using only 1,000 trajectories [Muennighoff et al., 2025]. We create three versions of the S1 dataset, built from examples with the shortest, longest, and random reasoning chains among several generations. Below we describe the data generation and finetuning processes (Section 5.1), and then describe our results and main findings (Section 5.2).

5.1 Data creation and finetuning setup

To construct the three variants of S1, we generate multiple responses for each S1 question-answer pair. Specifically, for each example, we produce 10 distinct answers using the QwQ-32B model, which we select for its superior performance among the evaluated models (Section 3). The generation process uses the hyperparameters detailed in Section 3.1. From these 10 responses per example, we derive three dataset variants—S1-short, S1-long, and S1-random—by selecting the shortest/longest/random response, respectively. This results in three datasets, each containing the same 1,000 queries but with distinct reasoning trajectories and answers. A histogram of the reasoning token counts for each dataset variant is provided in Appendix B.

Following the S1 approach, we finetune the Qwen-2.5-32B-Instruct model on the three S1 variants. The finetuning hyperparameters are consistent with those used for the S1.1 model [Muennighoff et al., 2025], and training is conducted on 32 H100 GPUs.⁵ The resulting models are evaluated using the benchmarks and experimental setup described in Section 3.1. Specifically, for each model we generate 20 answers per example, and report average accuracy.

⁵We match the number of gradient steps as in used for S1.1. Each model was finetuned for about 2 hours.

Table 3: Results for our finetuned models over the S1 variants: S1-short/long/random. We report average accuracy over 20 generations per example. The S1-short model improves performance over the other two models, while using fewer thinking tokens.

Metric	AIME 2024		AIME 2025		HMMT		Average	
	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑	Thinking Tokens ↓	Acc. ↑
S1-random	16145	68.8	17798	59.3	19243	40.8	17729	56.3
S1-long	16912	67.3	17973	58.5	19397	42.1	18094 (+2.1%)	56.0
S1-short	15364	68.3	17195	60.2	17557	45.2	16706 (−5.8%)	57.9

5.2 Finetuning results

Results are presented in Table 3. For the AIME 2025 and HMMT benchmarks, the S1-short variant achieves superior performance while using fewer thinking tokens. While performance on AIME 2024 is similar across models, S1-short still demonstrates the shortest thinking. Aggregated results across benchmarks reveal that the S1-short model improves relative performance by 2.8% compared to the S1-random baseline, with a reduction of 5.8% in thinking tokens. Conversely, the S1-long model consumes more tokens than S1-random, but obtains similar performance.

These results suggest that training on shorter reasoning sequences can lead to models that not only exhibit reduced computational overhead but also enhanced performance. This observation aligns with our findings in Section 3, which shows that answers with shorter thinking trajectories tend to be more accurate. We believe that developing models that reason more effectively with less computation holds significant potential, as supported by related work [Lu et al., 2025].

6 Limitations and broader impact

The primary limitation of the *short-m@k* method is the reliance on batch decoding, as it requires parallel generation of multiple reasoning trajectories. This dependency might restrict its applicability in scenarios where inference memory is constrained. It should be noted that *short-m@k* can be used without batch decoding, although its efficiency gains will be lower. Additionally, while we show that finetuning on shorter reasoning chains can improve performance and efficiency, our experiments are limited to a specific model (Qwen-2.5-32B-Instruct) and dataset (S1).

In terms of broader impact, this work holds promise for enhancing the efficiency and accessibility of reasoning-LLMs by significantly lowering the required computational resources and time. By reducing these barriers, the technology could become more widely available, thereby democratizing access to advanced reasoning capabilities across a broader range of users and applications. However, as is often the case with advancements in efficiency, the decreased cost and increased scalability also carry the risk of enabling wider misuse or unintended applications of these powerful models.

7 Conclusion

In this work, we challenged the common assumption that increased test-time computation leads to better performance in reasoning LLMs. Through empirical analysis on three complex mathematical and reasoning benchmarks, we showed that shorter reasoning chains consistently outperform longer ones, both in accuracy and computational efficiency. Building on this insight, we introduced *short-m@k*, an inference method that prioritizes early-terminating generations. *short-1@k*, our most efficient variant, is preferred over traditional majority voting in low-compute settings. *short-3@k*, while slightly less efficient, outperforms majority voting across all compute budgets. To further validate our findings, we fine-tuned an LLM on short reasoning trajectories and observed improved accuracy and faster runtime, whereas training on longer chains yields diminishing returns. These findings highlight a promising direction for developing faster and more effective reasoning LLMs by embracing brevity over extended computation.

Acknowledgments

We thank Miri Varshavsky Hassid for the great feedback and moral support.

References

- OpenAI. Learning to reason with llms, September 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.
- OpenAI. Openai o3-mini, 2025. URL <https://openai.com/index/openai-o3-mini/>. Accessed: 2025-02-24.
- Qwen Team. Qwq-32b: Embracing the power of reinforcement learning, March 2025a. URL <https://qwenlm.github.io/blog/qwq-32b/>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- Anthropic. Claude’s extended thinking, February 2025. URL <https://www.anthropic.com/news/visible-extended-thinking>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- Marah Abdin, Sahaj Agarwal, Ahmed Awadallah, Vidhisha Balachandran, Harkirat Behl, Lingjiao Chen, Gustavo de Rosa, Suriya Gunasekar, Mojan Javaheripi, Neel Joshi, et al. Phi-4-reasoning technical report. *arXiv preprint arXiv:2504.21318*, 2025.
- Akhiad Bercovich et al. Llama-nemotron: Efficient reasoning models, 2025. URL <https://arxiv.org/abs/2505.00949>.
- An Yang et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- Skywork open reasoner series. <https://capricious-hydrogen-41c.notion.site/Skywork-Open-Reasoner-Series-1d0bc9ae823a80459b46c149e4f51680>, 2025. Notion Blog.
- Google DeepMind. Gemini 2.5: Our most intelligent ai model, March 2025. URL <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/#gemini-2-5-thinking>.
- Qwen Team. Qwen3, April 2025b. URL <https://qwenlm.github.io/blog/qwen3/>.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Zixuan Ke, Fangkai Jiao, Yifei Ming, Xuan-Phi Nguyen, Austin Xu, Do Xuan Long, Minzhi Li, Chengwei Qin, Peifeng Wang, Silvio Savarese, et al. A survey of frontiers in llm reasoning: Inference scaling, learning to reason, and agentic systems. *arXiv preprint arXiv:2504.09037*, 2025.
- Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025.
- Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost. *arXiv preprint arXiv:2407.19825*, 2024.

- Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24312–24320, 2025.
- Xiao Pu, Michael Saxon, Wenyue Hua, and William Yang Wang. Thoughtterminator: Benchmarking, calibrating, and mitigating overthinking in reasoning models. *arXiv preprint arXiv:2504.13367*, 2025.
- Chenxu Yang, Qingyi Si, Yongjie Duan, Zheliang Zhu, Chenyu Zhu, Zheng Lin, Li Cao, and Weiping Wang. Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*, 2025.
- Zhaojian Yu, Yinghao Wu, Yilun Zhao, Arman Cohan, and Xiao-Ping Zhang. Z1: Efficient test-time scaling with code. *arXiv preprint arXiv:2504.00810*, 2025.
- Ximing Lu, Seungju Han, David Acuna, Hyunwoo Kim, Jaehun Jung, Shrimai Prabhumoye, Niklas Muennighoff, Mostofa Patwary, Mohammad Shoyebi, Bryan Catanzaro, et al. Retro-search: Exploring untaken paths for deeper and efficient reasoning. *arXiv preprint arXiv:2504.04383*, 2025.
- Mehdi Fatemi, Banafsheh Rafiee, Mingjie Tang, and Kartik Talamadupula. Concise reasoning via reinforcement learning. *arXiv preprint arXiv:2504.05185*, 2025.
- Daman Arora and Andrea Zanette. Training language models to reason efficiently. *arXiv preprint arXiv:2502.04463*, 2025.
- Junlin Wang, Shang Zhu, Jon Saad-Falcon, Ben Athiwaratkun, Qingyang Wu, Jue Wang, Shuaiwen Leon Song, Ce Zhang, Bhuwan Dhingra, and James Zou. Think deep, think fast: Investigating efficiency of verifier-free inference-time-scaling methods. *arXiv preprint arXiv:2504.14047*, 2025.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Mathematical Association of America. Aime 2024, 2024. URL https://artofproblemsolving.com/wiki/index.php/2024_AIME_I.
- Mathematical Association of America. Aime 2025, 2025. URL https://artofproblemsolving.com/wiki/index.php/2025_AIME_I.
- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL <https://matharena.ai/>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.
- Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. Spoc: Search-based pseudocode to code. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/file/7298332f04ac004a0ca44cc69ecf6f6b-Paper.pdf.
- Mark Chen et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>. arXiv:2107.03374.
- Michael Hassid, Tal Remez, Jonas Gehring, Roy Schwartz, and Yossi Adi. The larger the better? improved llm code-generation via budget reallocation. *arXiv preprint arXiv:2404.00725*, arXiv:2404.00725, 2024. URL <http://arxiv.org/abs/2404.00725>.

A Per benchmark results

We present the per-benchmark results for each of the criteria persented in Section 4.2. The sample-size (k) results are presented in Figures 6 to 8. The thinking compute comparison results are presented in Figures 9 to 11. The time-to-answer results per benchamrk are presented in Figures 12 to 14.

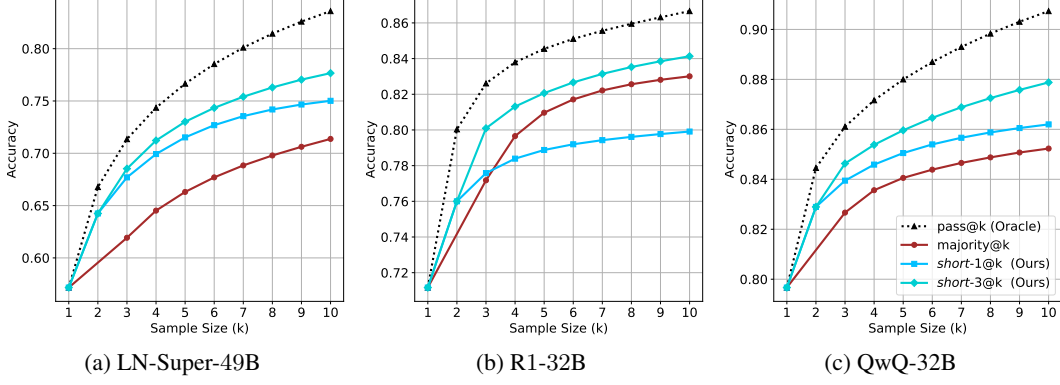


Figure 6: AIME 2024 - sample size (k) comparison.

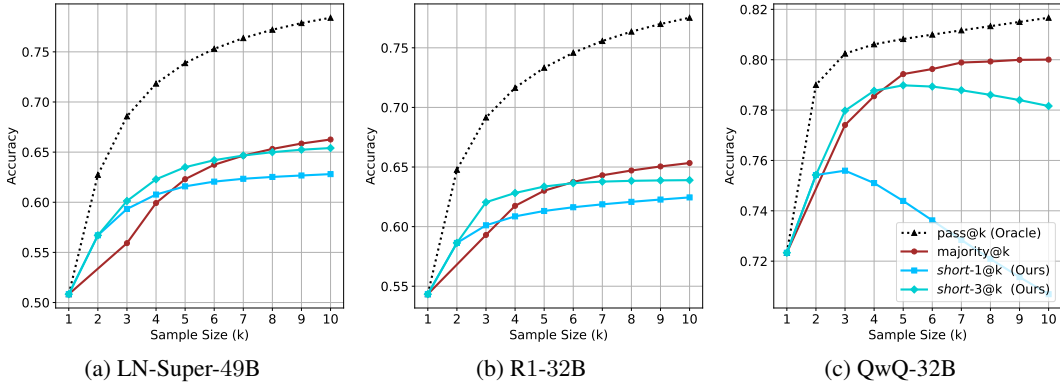


Figure 7: AIME 2025 - sample size (k) comparison.

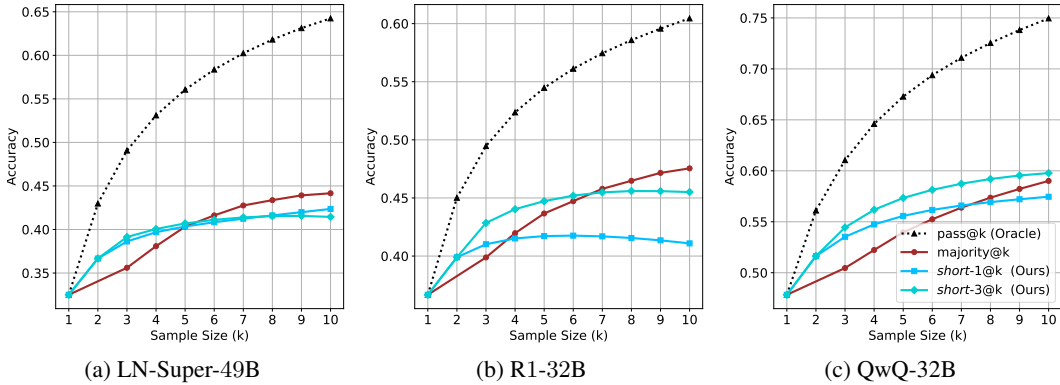


Figure 8: HMMT Feb 2025 - sample size (k) comparison.

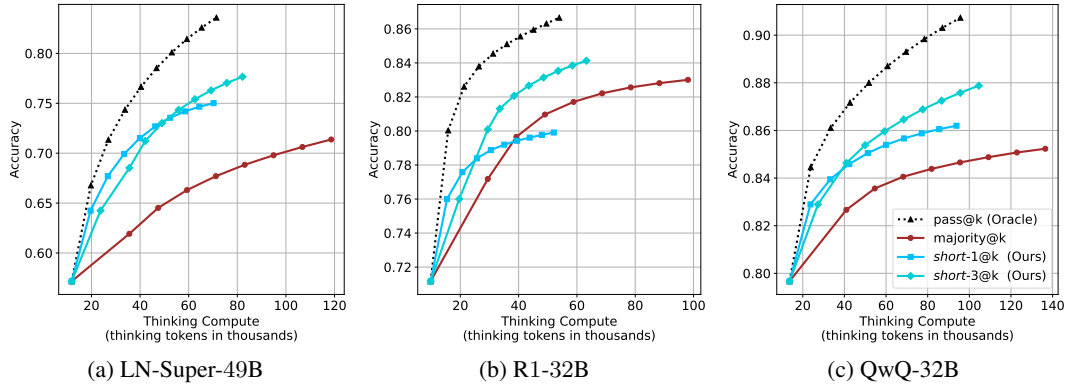


Figure 9: AIME 2024 - thinking compute comparison.

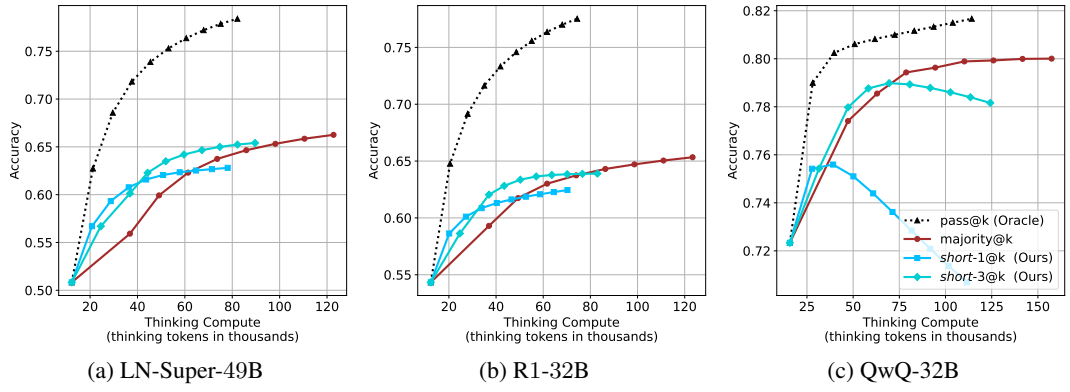


Figure 10: AIME 2025 - thinking compute comparison.

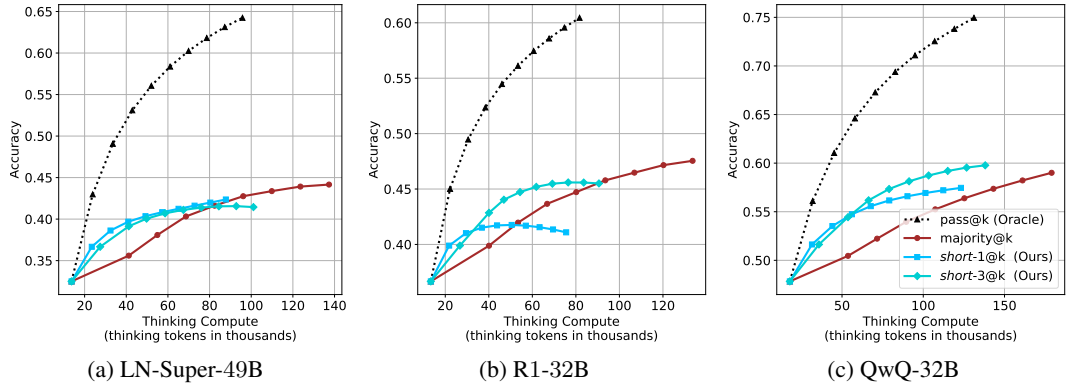


Figure 11: HMMT Feb 2025 - thinking compute comparison.

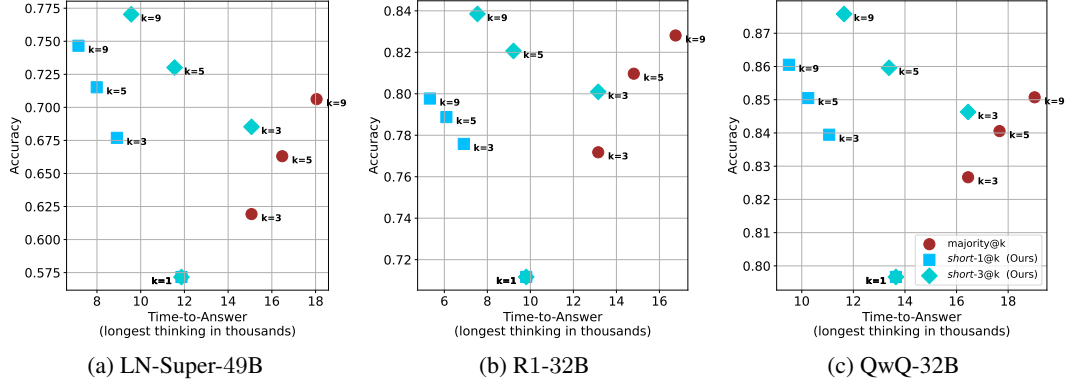


Figure 12: AIME 2024 - time-to-answer comparison.

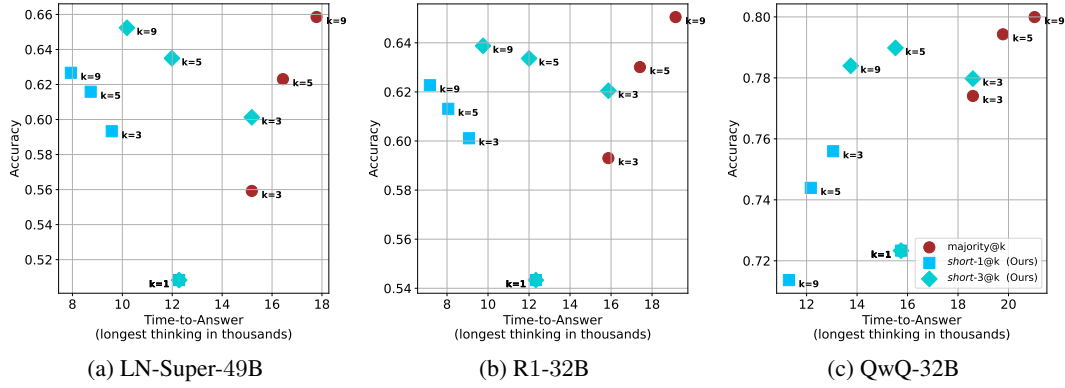


Figure 13: AIME 2025 - time-to-answer comparison.

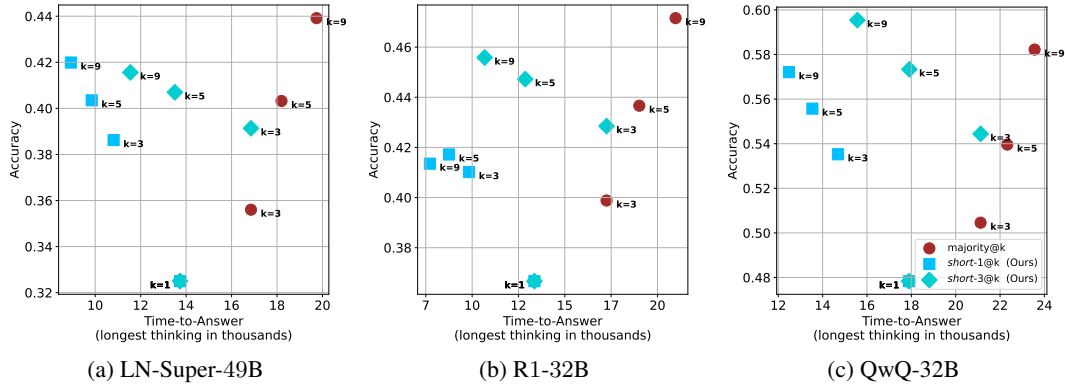


Figure 14: HMMT Feb 2025 - time-to-answer comparison.

B Thinking token histograms of S1 variants

Figure 15 shows the thinking token count histograms for the three variants of the S1 dataset (short/-long/random) presented in Section 5.

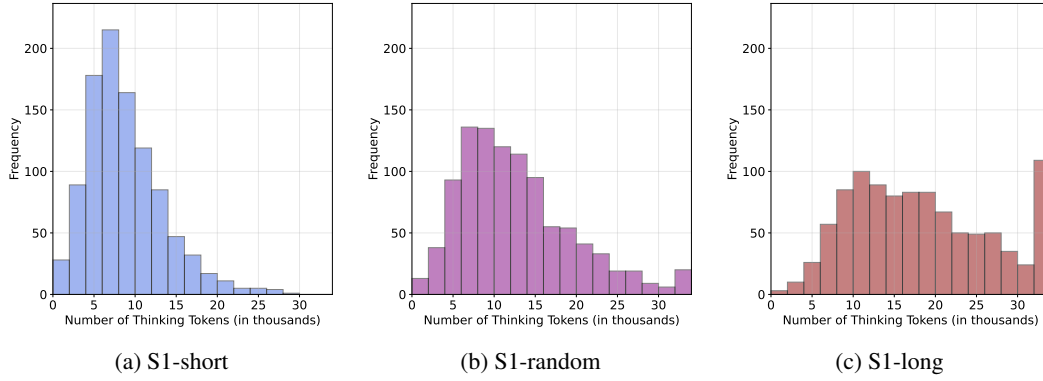


Figure 15: Thinking token count histograms for S1-short, S1-random and S1-long datasets.